CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT — 6

Name: Hari krishan	UID: 24BA170898
Section/Group: 24AIT NTPP 3 (A)	Branch/Semester: CSE AIML (CS221) / 5 th
Subject Code: 24 CSP-337	Subject Name: Full Stack - Il
Date of Performance:	Submitted to: Mr.Rakshit Ajay

Experiment — 6.1

l) AIM: To implement paginated and sortable REST APIs for efficient and scalable data retrieval

2) OBJECTIVE

I . To understand pagination and sorting concepts
2. To implement pageable APIs using Spring Data
3. To optimize data retrieval for large datasets
4. To improve API performance and usability

3) SOFTWARE REQUIREMENT

l Java(JDK 17)
2. Spring Boot
3. IDE(VSC0dc/Eclipse)
4. Postman/API testing tool
5. Mysql workbench

4) THEORY

In real-world applications, datasets can grow significantly large, making it inefficient to retrieve all records in a single API call. Pagination addresses this issue by dividing data into smaller chunks (pages), allowing clients to request only a subset of data at a time.

Sorting enhances usability by allowing data to be ordered based on specific fields such as creation date, popularity, or relevance.

Spring Boot provides built-in support pagination and sorting through the Pageable interface, which simplifies API design and ensures efficient query execution.

Benefits of pagination and sorting include:

- Reduced server load
- Faster response times • Improved user experience
- Better scalability

5) IMPLEMENTATION/PROCEDURE

l. Implement (iET APIs with pagination support
2. Add sorting parameters to APIs

3. Configure default page size and sorting
4. Optimize response payload

6) CODE

Database queries

```
CREATE TABLE products ( id INT PRIMARY
  KEY AUTO INCREMENT, name
  VARCIIAR(100) NOT NULL, category
  VARCHAR(50) NOT NULL, price
  DECIMAL( 10, 2) NOT NULL

INSERT INTO products (name, category, price) VALUES
('Gaming Laptop', 'Electronics',
150000.00), ('Bluetooth Speaker'.
'Electronics', 5()0().()()), ('Wireless
Keyboard', 'Electronics', 3000.00), ('4K
Monitor', 'Electronics', 60000.00),
('Smartwatch', 'Electronics', 1 5000.00),
('Tablet', 'Electronics', 25000.00),
('Mouse Pad', 'Accessories', 700.00),
('Portable Charger', 'Accessories',
3500.00), ('Webcam HD, 'Electronics'.
5500.00),
('Laser Printer', 'Electronics', 25000.00),
('Ergonomic Office Desk', 'Furniture'. 25000.00),
('Ergonomic Office Chair', 'Furniture', 2()()00 00),
('File Cabinet', 'Furniture', 15000.00),
('Sticky Notes', 'Stationery', 50.0()),
('Marker Set', 'Stationery, 400.00),
('Travel Backpack', 'Accessories',
35()0.()()), ('LEI) I)esk Lamp',
'Furniture'. 2500.00).
('External SSD', 'Storage', 12000.00),
CNetworkAttached Storage', 'Storage', 45000.00),
('USB Hub', 'Accessories', 1500.00),
('Smart TV', 'Electronics', 80000.00),
```

('Wireless Earbuds', 'Electronics', 7000.00),
('Mechanical Keyboard', 'Electronics', 5000
00), ('Gaming Chair', 'Furniture', 25000.00),
('Office Desk', 'Furniture', 30000.00), ('Paper
Clips'. 'Stationery', 1 00.00), ('Highlighter
Set', 'Stationery', 600.00), ('Suitcase',
'Accessories', 5000.00), ('Desk Organizer',
'Furniture', 1500.00), ('External HDD',
'Storage', 7000.00),
('Wireless Router', 'Electronics', 6000.00),
('Smart Door Lock', 'Electronics', 25000.00),
('Desk Pad', 'Furniture'. 1800.00).
('Notebook Set', 'Stationery',
500.00), ('Pen Holder', 'Furniture',
800.00),
('Gaming Mouse', 'Flectronics'. 4000.00).
('Wireless Charger', 'Electronics'. 3500.00), (
Printer Ink Cartridge', 'Electronics', 2000.00),
CProjection Screen'. 'Electronics', 40000.00),
('Office Fan', 'Fumiture', 3000.00),
('USB Flash Drive 64GB', 'Storage', 1500 00),
('Bluetooth Ileadphones'. 'Electronics', 1 20()().()()),
('Laptop Stand', 'Furniture', 2500.00),
('Cordless Drill', 'Tools', IOOOO.OO),
('Electric Screwdriver'. 'Tools',
8000.00), ('Paint set', 'Tools', 3500.00),
('Garden Shovel', 'Tools', 2000.00),
('Lawn Mower', 'Tools', 45000.00),
('Hose Pipe', 'Tools', 2500.00),
('Car Vacuum Cleaner', 'Automotive',
3500.00), ('Jump Starter', 'Automotive',
15()00.()()). ('Car Cover', 'Automotive',
5000.00), ('Motor ()il 5L', 'Automotive',
200().()0), ('Air Compressor', 'Tools',
25000.00),
('Bike Ilelmet', 'Automotive', 3000.00).
('Roadside Emergency Kit', 'Automotive', 4000.00);

Database configuration sprmg.application.name—pagination-api

spring.datasource.url=jdbc:mysql://localhost:3306/pagination
db spring.datasource.username=root
spring.datasource.password mypassword

spring.jpa.hibernate.ddl-auto none
spring.jpa.show-sql true
spring.jpa.properties.hibernate.fbrmat
sql=true

UNIVERSITY
Discover. learn. Empower.

Model: Product.java package
com.example.pagination api.model• import
jakarta.persistence Entity;

Import jakarta.persisten ce. Id•

Import jakarta.persisten ce. Table;

@Entity
@Table(name_"
products ")
public class Product {

private Long id;

private String name;
private String category;
private double price;

public Product() { public Product(Long id, string name, string

categow, double price) {

this.name name; this.
category = category;
= this.price
price;

public Long getJd() {
return id;

```
public void setld(lxmg id) {
    this.id — id;

public string getName() {
  return name,
        public void setName(String name) {
          this.name ¯ name;

        public String getCategory() {
          return category;

        public void setCategory(String category) {
          this.category category;

        public double getPrice() {
          return price;

        public void setPrice(double price) {
          this.price price;
```

Repository: ProductRcpository.java package com.example.pagination api.reposit01•y; import com.example.pagination api.model.Product; import org.springframework.data.jpa.repository.JpaRepository; public interface ProductRepository extends JpaRepository<Product,Long> I

Service: ProductService.java package com.example.pagination api.service•, import com.example.pagination api.model.Product; import com.example.pagination

api.repository.ProductRepository; import
org.springframework.data.domain.Page; import
org.springframework.data.domain.Pageable; import
org.springframework.stereotype.Service:

@Service public class ProductService {private final ProductRepository productRepository; public ProductService(ProductRepository productRepository) {this. productRepository productRepository;

```
public Page<Product> getAllProducts(Pageable pageable) {
  return productRepository. findAll(pageable);
```

Dto: PageResponse.java

package com.example.pagination api.dto; inoport java.util.List; public class PageResponse<T> { private List<T> data; private int currentPage; private int pageSize; private long totalElements; private int totalPages; private boolean first; private boolean last;

```
public PageResponse(
      List<T> data, int
      currentPage, int
      pageSize, long
      totalElements, int
      totalPages,
      boolean first,
      boolean last) {

    th is. data data:
     this. curren tPage currentPage;
     this.pageSi7.e = pageSi7.e; this.
     totalElements totalElements; this.
     totalPages = totalPages:
    this. first first:
    this.last = last;
```

```
public              getData() {
  return data,

public  int  getCurrentPage()  {
  return currentPage;

public  int  getPageSize()  {
  return pageSize•,

public  long  getTotalElemcnts()  {
  return totalElements:

public  int  getTotalPages()  {
  return totalPages;

public  boolean  isFirst()  {
  return first,

public  boolean  isLast()  {
  return last;
```

Dto:ErrorResponse.java

```
package com.example.pagination api.dto;

public class IErrorResponse {
  private  String  message;  public
  ErrorResponse(String  message)  {
  this.message¯message;
```

```
    public String getMessage() {
        return message;

    public void setMessage(String message) {
        this. message=message;
```

Controller: ProductController.java package com.example.pagination api.controller:

```
import com.example.pagination api.model.Product:
import com.example.pagination api.service.ProductService;
import com.example.pagination api.dto.PageResponse;
import com.example.pagination api.dto.ErrorResponse;

import org.springframework.data.domain.Page;
import
org.springframework.data.domain.PageRequest;
import org.springframework.http.ResponseEntity;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("%/api/products") public
class ProductController { private final
ProductService productService;

    public ProductCon trol ler(ProductService productService) {
        this.productService ¯ productService;


    @GetMapping
    public ResponseEntity<'?> oetAllProducts(
            @RequestParam(dcfaultValuc "0") int page,
            @RequestParam(defaultValue — "5") int size,
```

```
    @RequestParam(defaultValue — "id") String sortBy.
    @'RequestParam(defaultValue ¯ "asc") String direction) {

// Validate page if (page < 0) { return
ResponseEntity. badRequest()
        .body(new ErrorResponse("Page number cannot be negative. "));


// Validate page size if (size < Isize >
60) { return ResponseEntity.
badRequest()
        .body(new ErrorResponse("Page size must be between I and 60."));


// Validate sorting field
if !sortBy.equals("id") &&
    !sortBy.equals("name") && !
    sortBy.equals("category")
    &&
    !sortBy.equals("price")) {

    return ResponseEntity.badRequest()
     .body(new ErrorResponse("Invalid sort field. Use id, name, category. or price.
                                                                    "));


// Validate sorting direction
if !direction.equalsIgnorcCasc("asc") && !
   direction.equalsIgnorcCasc("desc")){

    return ResponseEntity. badReques1()
        .body(new ErrorResponse("Sort direction must be asc or desc."));


Sort.Direction sortDirection _direction.
      equalsIgnoreCase("desc")
       ? Sort.Direction.DESC
       :Sort. I)irection.ASC;
```

```
PageRequest pageable=
    PageRequest.of(page, size, Sort.by(sortDirection, sortBy)):

Page<Product> result productService.getAllProducts(pageable):

PageResponse<Product>  response  _  new
    PageResponse<>(  result.getCon  tent(),
    result.getNumber(), result.getSi7.e(),
     result.getTotalElements(),
     result.getTotalPages(),
     result.isFirst(),
     result.isLast()

return ResponseEntity.ok(response);
```

Web configuration with frontend(CORS)

```
package com.example.pagination api.conflg;

import org.springframework.context.annotation.Configuration; inoport
org.springframework.web.servlet.config.annotation.CorsRegistry; import
org.springframework.web.servlet.con fig. annotation. WebM vcC on figurer;

(a) Con figuration public class WebConiig implements
WebMvcConfigurer {

(  a,  Override  public  void
addCorsMappings(CorsRegistry registry) {

  registry.addMapping("/**")
      .allowedOrigins("http://localhost:_3000")
       .allowedMethods("GET","POST", "PUT' "DELETE' "OPTIONS")
      .allowedHeaders("*");
```

7) OUTPUT

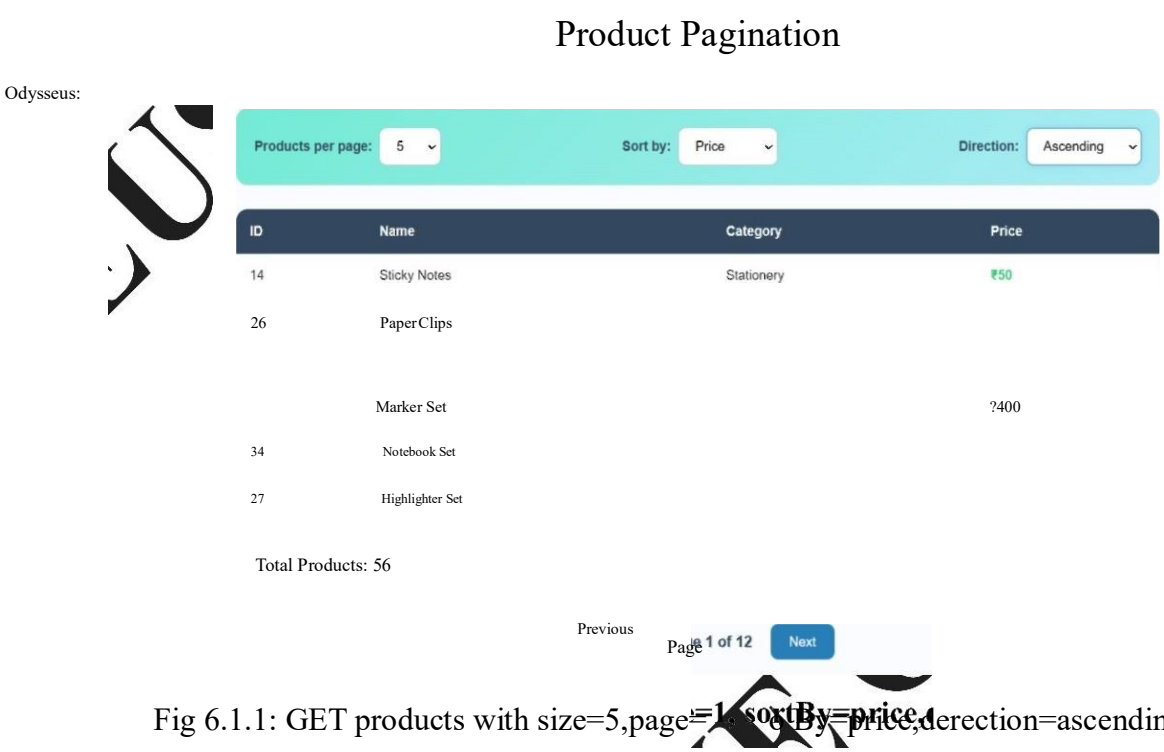


Fig 6.1.1: GET products with size=5,page=1, sortBy=price,derection=ascending(asc)

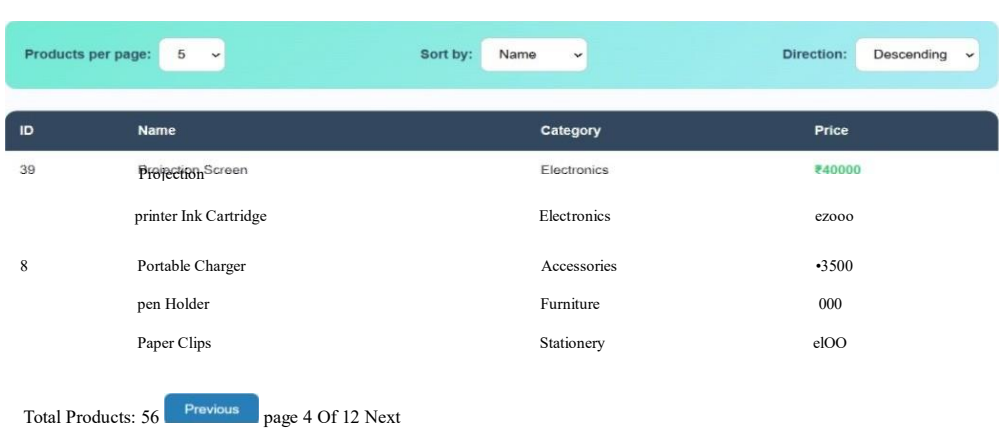


Fig 6.1.2: GET products with size=5,page=4, sortBy=name,derection=descending(desc)

8) LEARNING OUTCOMES

- Designed and implemented REST ful APIs using Spring Boot for efficient retrieval of data
- Implemented

Implemented pagination to retrieve data in manageable pages and reduce unnecessary database and network load

- Implemented dynamic sorting on different fields and sort directions
- Applied input validation to ensure valid page numbers, page sizes, sorting fields and directions
- understood the role of spring Data JPA and pageable in efficient database-driven data retrieval
- Developed scalable API endpoints capable of handling large datasets without retrieving all records at once.
- Integrated backend APIs with a frontend application to support paginated and sortable data representation

Experiment 6.2

1) AIM: To optimize backend read operations using caching and efficient database query strategies.

2) OBJECTIVE

1 . To identify and resolve inefficient database queries
2. To understand the N+l query problem
3. To implement caching mechanisms
4. To improve API performance through optimization

3) SOFTWARE REQUIREMENT

l .Spring Boot

2

Database(MYSQL/PostgreSQL) 3

Jmeter for optimization testing

4. Caffeine in-memory caching

4) THEORY

Backend performance is critical in applications that handle large volumes of read requests. Inefficient queries and repeated database access can significantly degrade performance.

One common issue is the N+l query problem, where fetching related entities results in multiple database queries instead of a single optimized query. This leads to increased latency and unnecessary database load.

This problem can be resolved using techniques such as JOIN FETCH, which retrieves related data in a single query, reducing database round trips.

Another key optimization technique is caching, where frequently accessed data is stored m Inemory to avoid repeated database queries. Tools like Ehcache enable efficient caching strategies, improving response times and reducing system load.

For complex queries, native SQL queries can be used to gain better control over execution and performance.

Finally, performance benchmarking using tools like Apache JMetcr helps evaluate system performance under load and validate the effectiveness of optitnization strategies.

5) IMPLEMENTATION/PROCEDURE

l .Identity inefficient queries

2. Apply JOIN FETCH to optimize queries
3. Implement caching for frequently accessed data
4. Write native SQL queries for complex operations 5 Benchmark API performance using JMeter

6) CODE

Database queries

```
CREATE TABLE suppliers ( id BICJINT
  PRIMARY KEY, name
  VARC11AR(255) NOT NULL
```

```
ALTER TABLE products ADD
COLUMN supplier id BIGINT;
ALTER TABLE products
ADD CONSTRAINT Ik_product supplier
FOREIGN KEY (supplier id)
REFERENCES suppliers(id)•,
INSERT INTO suppliers (id, name)
VALUES
( l , 'ABC Suppliers'),
(2, 'Global Traders'),
(3, 'Premium Distributors');
/*mapping suppliers to products*/
UPDATE products
SET supplier id - CASE
  WHEN id I THEN I
  WHEN id 0 0 -3 = 2 THEN 2
  WHEN id % THEN 3
ENI)
WHERE O;
```

Updated application.properties

```
spring. application .name=pagination -api
spring.datasource.url¯jdbc:mysql://localhost:3306/pagination
db spring.datasource.username¯root
spring.datasource.password¯mypassword4
spring.jpa.hibernate.ddl-auto=none
# Show SQL queries spring.jpa.show-
sql=true
# Format SQL for readability
spring.jpa.properties.hibernate. format sql=true
#show sql parameters logging.level.org.hibernate.orm.jdbc.bind—
TRACE

spring.cache.type=caffeine spring. cache. cache-names=products
spring.cache. camine.spcc=maximumSize— I )0,expircAfterWrite—
1 0m.

#jmeter monitoring management. endpoints. web. exposure.
include=health,info,metrics
management. endpoint. h ealth. sh ow-detai Is=a I ways
```

added dependencies to pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-cache</artifactId>
</dependency>

<dependency>
  <groupId>com.github.ben-manes.caffeine</groupId>
  <artifactId>caffeine</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>  <artifactId>spring-boot-starter-
  actuator</artifactId>
</dependency>
```

CacheConfig.java

```
package com.example.pagination api.contig;

import org.springframework.cache.CacheManager; import
org.springframework.cache.caffeine.CaffeineCacheManager;
import org.springframework.context.annotation.Bean; import
org.springframework.context.annotation.Configuration;

@OConfiguration public
class CacheConfig {
  @Bean
  public CacheManager cacheManager() {
    CaffeineCacheManager cacheManager= new CaffeineCacheManager("products");
    return cacheManager;
```

Update ProductService.java by adding impon
org.springframework.cache.annotation.Cacheable; and
above the GetAllProducts() add

```
@Cacheable(
    value="products", key="#pageable.pageNumber + - + #pageable.pageSize +
    - + #pageable.sort"
```

Supplier.java package com.example.pagination api.model:

```
ilnport jakarta.persi stence.Entity:
import jakarta.persi stence.ld•,
import jakarta.persi stence. Table;
@Entity
@Table(name 'suppliers")
public class Supplier {

  private Long id:
  private String name:
  public Supplier(){

  public Supplier(Long id, String
    name){this.id—id;
    this.name=name;

  public Long getId(){
    return id;

  public void setld(Long id){ this.id—id;

  public String getName() {
    return name:

  public void setName(String name) {
    this.name=name:
```

Update product.java

```
import jakarta.persisten ce. ManyToOne;
import jakarta.persistence..IoinColumn:
 import jakarta.persistence.FetchType; then
added this to the Product class
@ManyToOne(fetch = FetchType.LAZY)
  @JoinColumn(name 'supplier id")
```

```
    private Supplier supplier;

    public    Supplier    getSupplier(){
        return supplier:
    public          void          setSupplier(Supplier          supplier){
        this.supplier¯supplier;
```

SupplierRepository.java package com.example.pagination api.repository;

```
import com.examplc.pagination api.nnodel.Supplier; import
org.springframework.data.jpa.repository.JpaRepository;

public interface SupplierRepository extends JpaRepository<Supplier,Long>{
```

N+l query implementation: updated ProductSevice.java

```
package com.example.pagination api.service; import
com.examplc.pagination api.modcl.Product; import
com.example.pagination api.repository. ProductRepository;
import org.springframework.data.domain.Page; import
org.springiiamework.data.domain.Pageable; import
org.springframework.stereotype.Service; import
org.springframework.transaction.annotation.Transactional:
@Service public class ProductService {private final
ProductRepository productRepository; public
ProductService(ProductRepository productRepository) {
this.productRepository productRepository;


    @ül'ransacti onaI(readOnly true) public Page<Product>
    getAllProducts(Pageable pageable) {
  Page<Product> products = productRepository.f1ndAll WithSupplier(pageable):

        // Access supplier for every product
        products.getContent().forEach(product -> {
            if (product.getSupplier() != null) {
            if (product.getSupplier() != null) {
```

```
            System.out.println(
              "Product: " product.getName()
              + " Supplier.„
              + product.getSupplier().getName()
            );
        }
    });
    return products;
```

Update ProductRepository.java for JOIN FETCH query

```
package com.example.pagination api.repository; import
com.example.pagination api.model.Product; import
org.springframework.data.domain.Page; import
org.springframework.data.domain.Pageable; ilnpolt
org.springframework.data.jpa.repository.JpaRepository; import
org.springframework.data.jpa.repository.Query; public interfäce
ProductRepository extends JpaRepository<Product,Long> {
  @Query(
  value — tt tt tt
    SELEC p
    FROM Product p
    JOIN FETCH p.supplier

  countQuery= """
    SELECT COUNT(p)
    FROM Product p

  Page<Product> findAIIWithSupplier(Pageable pageable);
```

In productService.java change Page<Product> products ¯ productRepository.fmdAll(pageable); to Page<Product> products _ productRepository. findAllWithSupplier(pageable);

7) OUTPUT

Product Pagination/optimized backend reads

Created by Anye Odysseus: 24BA170992

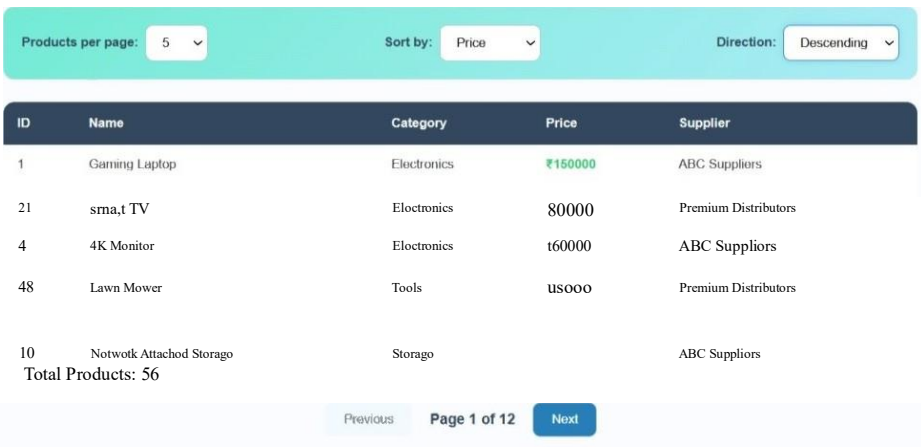


Fig 6.2.1: GET products with supliers(join fetch query)

Performance tests

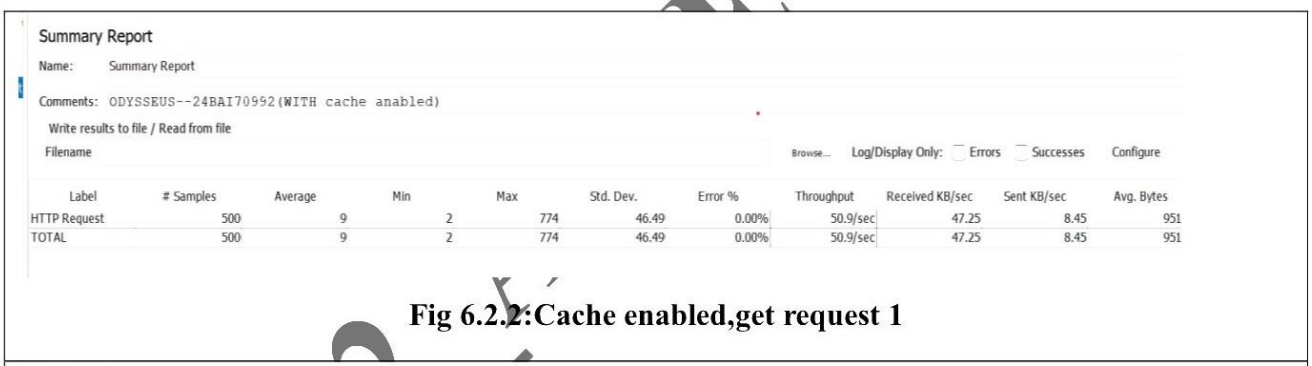


Fig 6.2.2:Cache enabled,get request 1

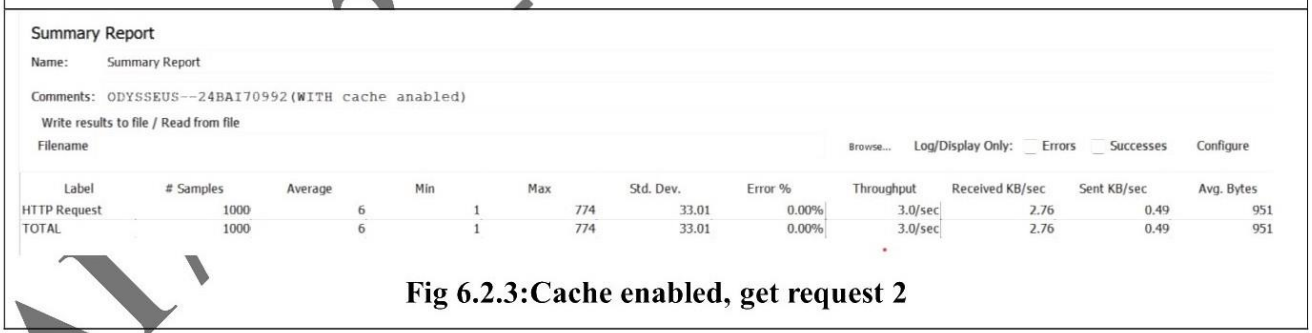


Fig 6.2.3:Cache enabled, get request 2

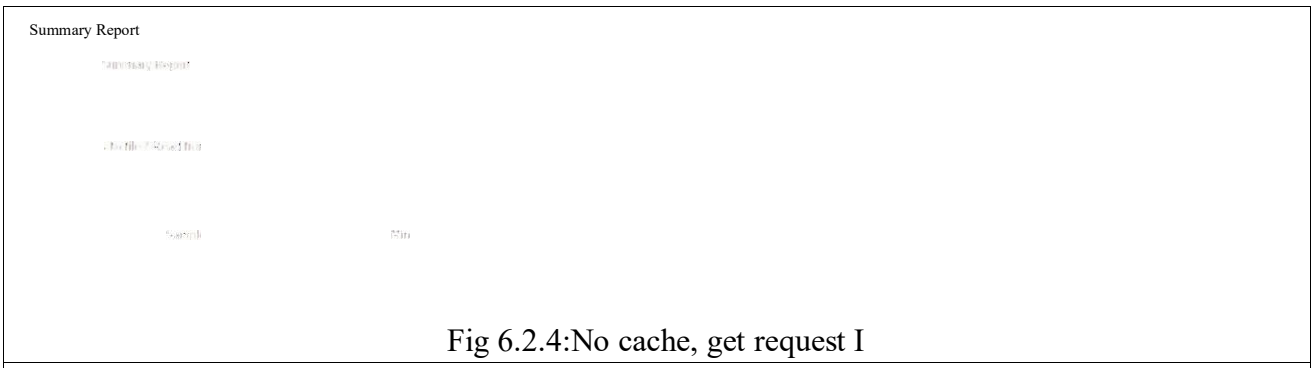


Fig 6.2.4:No cache, get request I

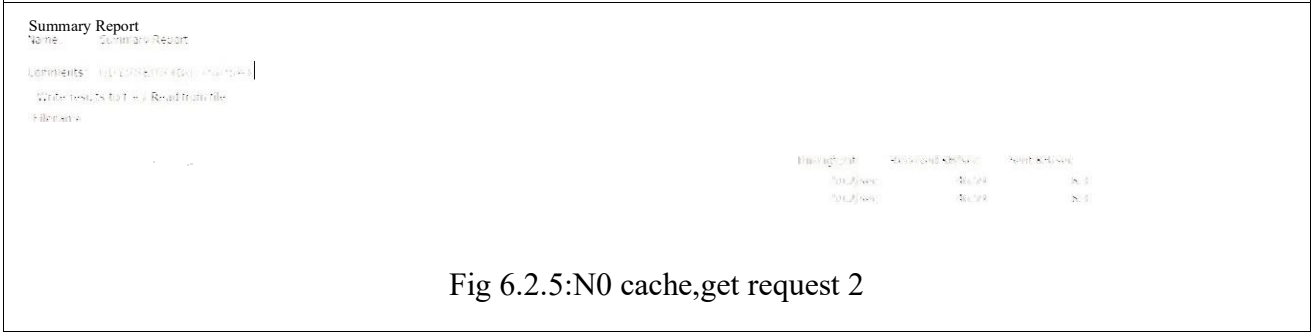


Fig 6.2.5:N0 cache,get request 2

8) LEARNING OUTCOMES

- Identified and understood the N+l query problem in JPA-based applications
- Use JOIN FETCH to optimize database queries and efficiently retrieve related entities
- Implemented caching using Caffü2ine and Spring Cache to reduce repeated database access.
- Understood cache hits, cache misses, expiration, and cache keys in backend applications
- Analyze and compare database query behavior betore and after query optimization
- Measure API performance using JMeter and evaluate response time, throughput, and error rate
- Applied backend optimization techniques to improve API performance, scalability, and efficient resource utilization